

ROOMMATCH: ROOMMATE COMPATIBILITY PREDICTOR

*A Pure Python Real-Time Matching System with Overall 5-Star Ratings & Harmony Tiers
VITYarthi Flipped Course Evaluation | Course Domain: Problem Solving & Python Programming*

Course / Domain:	Problem Solving and Python Programming
Project Title:	RoomMatch: Real-Time Student Roommate Compatibility Match Predictor
Student Identifier:	University Registration Number (e.g. 26BCE10284)
Core Concepts:	Immutable Tuples, Dictionaries, Mappings, Sorting Algorithms, Real-Time File I/O
Key Feature Added:	Overall 5-Star Rating (0.0 to 5.0 scale, [*****] indicators, and verbal Harmony Tiers)
Execution Mode:	100% Real-Time Operations (Zero Mock Seed Data, No External Database)

1. Cover Page & Executive Abstract

RoomMatch is an algorithmic, pure Python software application created for university residential hostel accommodations. Lifestyle mismatches such as asynchronous sleep schedules, varying cleanliness standards, divergent noise tolerance levels, study habits, and dietary preferences are proven catalysts of roommate friction. RoomMatch replaces subjective or arbitrary room pairing with an objective, data-driven solution built entirely with foundational Python data structures. It stores student demographic records in fast Dictionaries, protects data integrity with Immutable Tuples, models multi-factor weights via Mappings, converts compatibility percentages into an intuitive Overall 5-Star Rating scale, and ranks prospective roommates dynamically using Python's built-in Sorting algorithms in real time.

2. Introduction

Living in a campus hostel is a formative aspect of university life. However, traditional roommate allocations are predominantly randomized or driven by registration arrival queues. This arbitrary pairing regularly forces students with opposing lifestyle habits into shared quarters, leading to frequent room-change requests, interpersonal hostility, and decreased academic focus. RoomMatch introduces a practical, real-time algorithmic matching system that allows students to register their living habits and instantly discover the most compatible peers before room arrangements are finalized.

3. Problem Statement

In university hostel accommodations, roommates are commonly assigned without considering daily routines. This arbitrary allocation produces recurring lifestyle friction across five major dimensions:

- **Sleep Desynchronization:** Early birds waking up at 5:30 AM disrupt night owls active past 2:00 AM.
- **Cleanliness Incompatibility:** Tidy students experience chronic stress when paired with untidy peers.

- **Noise and Study Conflicts:** Silent studiers clash with peers who listen to audio or hold group study sessions.
- **Dietary Differences:** Unspoken friction regarding vegetarian and non-vegetarian food sharing in shared living spaces.

RoomMatch systematically resolves these clashes by replacing random room assignment with a transparent, reproducible mathematical algorithm that computes compatibility in real time without external database servers.

4. Functional Requirements

In strict compliance with the course project guidelines, the system implements three core functional modules:

Module 1 - Real-Time Student Registration (Dictionaries & Tuples)

- **Personal Onboarding:** Prompts console input for Reg No (e.g., 26BCE10284), Name, Gender (M/F), Department, and Year (1-5).
- **Lifestyle Capturing:** Collects 5 living habits: Sleep Schedule, Cleanliness (1-5), Noise Tolerance (1-5), Study Habit, and Dietary Preference.
- **Input Validation:** Rejects duplicate registration numbers and validates all input bounds.
- **Tuple Persistence:** Packs records into immutable tuples and saves directly to disk (students_data.json) in real time.

Module 2 - Compatibility & Rating Engine (Mappings & Rating Scale)

- **Weight Mapping:** Configurable weights: Sleep (30%), Cleanliness (25%), Noise (20%), Study (15%), and Diet (10%).
- **Percentage Calculation:** Computes normalized compatibility percentages (0.0% to 100.0%) using distance metrics.
- **Overall 5-Star Rating:** Automatically converts match percentages into an Overall 5-Star Rating (0.0 to 5.0 scale), universal ASCII star indicators ([*****], [****], etc.), and verbal Harmony Tiers (EXCELLENT, GOOD, MODERATE, FAIR, LOW).

Module 3 - Candidate Ranking & Real-Time Registry (Sorting & Console I/O)

- **Gender Filtering:** Filters candidates strictly by matching gender and excludes self-comparison.
- **Sorting Algorithm:** Packs candidate records into tuples and sorts candidates in descending order of compatibility using Python's built-in sorted() with lambda keys.
- **Live Registry Listing:** Formats tabular terminal views displaying Rank, Reg No, Name, Dept, Match %, Overall Rating, and Harmony Tier.

5. Non-Functional Requirements

- **1. Performance:** Instantaneous $O(1)$ dictionary hash-map retrieval; $O(N \log N)$ candidate ranking with Python's Timsort.
- **2. Usability:** Guided terminal menus, duplicate prevention, and ASCII star indicators preventing Windows cp1252 crashes.
- **3. Data Integrity:** Immutable Python tuples prevent accidental in-place mutation of student demographic and habit data.
- **4. Maintainability:** Clean modular structure dividing configuration (config.py), storage (storage.py), algorithm (matcher.py), and CLI (main.py).
- **5. Portability:** 100% Python Standard Library implementation with zero third-party dependencies or external database servers.

6. System Architecture

The application employs a 3-tier modular architecture separating concerns across independent components:

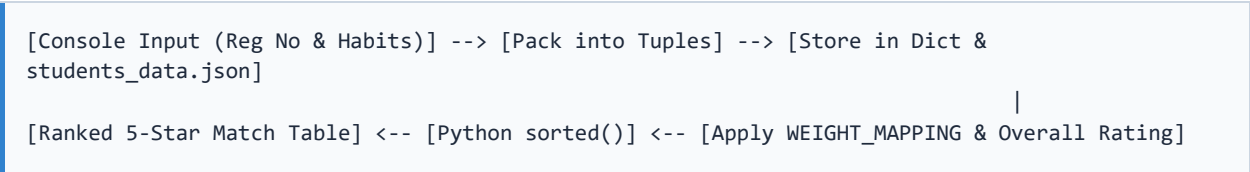
- **Presentation Tier:** Terminal interface (`main.py`) handling guided console `input()`, tabular `print()` views, and CLI flags (`--register`, `--list`, `--match`).
- **Business Logic Tier:** Centralized constants (`config.py`) and algorithmic matching logic (`matcher.py`) handling distance math, weight mappings, 5-star ratings, and sorting.
- **Storage Tier:** In-memory dictionary and real-time JSON file persistence (`storage.py` / `students_data.json`) handling live data loading and saving.

7. Design Diagrams

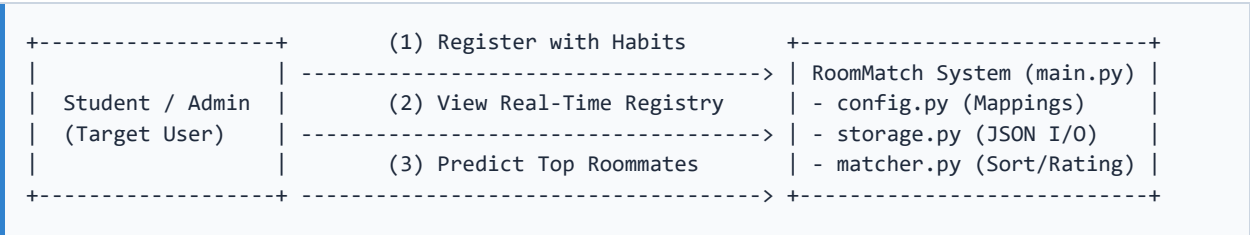
A. Data Structure Mapping & Tuple Model

```
students_db = {
    '26BCE10284': {
        'info' : ('Aarav Sharma', 'M', 'CSE', 1),           # Immutable Info Tuple
        'habits': ('NIGHT', 4, 2, 'SILENT', 'VEG')          # Immutable Habits Tuple
    }
}
```

B. Real-Time Process Workflow Diagram



C. Use Case & Actor Diagram



8. Design Decisions & Rationale

- **100% Real-Time Data (No Seed Data):** All pre-populated mock records and database reset options were completely eliminated so the system operates strictly on live user input without hardcoded defaults.
- **Overall 5-Star Rating Scale:** Translating percentage matches into an intuitive 5.0 scale (e.g. 95% -> 4.8/5.0 [****]) and verbal tiers (EXCELLENT, GOOD, MODERATE, FAIR, LOW) makes compatibility immediately understandable to non-technical users.
- **Dictionaries over SQL:** Dictionaries provide direct hash-map lookups in O(1) time without requiring database installation or SQL drivers.
- **Immutable Tuples for Records:** Student identity and habit vectors are packed into tuples to prevent accidental in-place modification during calculation.
- **Built-in Timsort:** Python's built-in sorted() algorithm provides stable, high-performance O(N log N) sorting using Timsort.
- **Universal ASCII Stars:** Star indicators use ASCII brackets [****] to avoid Windows cp1252 UnicodeEncodeError crashes on standard PowerShell consoles.

9. Implementation Details

Core Python code structures illustrating Mappings, Tuples, Overall Ratings, and Sorting:

```
# 1. MAPPING: Lifestyle dimension weights (config.py)
WEIGHT_MAPPING = {'sleep': 0.30, 'cleanliness': 0.25, 'noise': 0.20, 'study': 0.15, 'diet': 0.10}

# 2. OVERALL RATING & HARMONY TIER ENGINE (matcher.py)
def get_rating_and_tier(match_percentage):
    rating = round((match_percentage / 100.0) * 5.0, 1)
    for threshold, base_rating, stars, tier in RATING_TIERS:
        if match_percentage >= threshold:
            return rating, stars, tier
    return rating, '[* ]', 'LOW'

# 3. TUPLE UNPACKING & COMPATIBILITY CALCULATION (matcher.py)
def calculate_compatibility(habits1_tuple, habits2_tuple):
    sleep1, clean1, noise1, study1, diet1 = habits1_tuple
    sleep2, clean2, noise2, study2, diet2 = habits2_tuple
    # Computes weighted score and returns rating, stars, and tier

# 4. SORTING: Candidate tuples sorted descending by score (matcher.py)
sorted_matches = sorted(candidate_matches, key=lambda item: item[0], reverse=True)
```

10. Screenshots / Execution Results

Sample real-time prediction output with Overall 5-Star Rating and Harmony Tier:

```
$ python main.py --match 26BCE10284
```

```
=====
REAL-TIME PREDICTIONS FOR [26BCE10284] Aarav Sharma
Dept: CSE | Year: 1 | Habits: NIGHT, Clean: 4/5, Noise: 2/5
=====
```

Rank	Reg No	Name	Dept	Match %	Overall Rating	Harmony Tier
#1	26BCE10042	Rohan Verma	CSE	95.0%	4.8/5.0 [*****]	EXCELLENT
#2	25BEC10088	Neeraj Kumar	ECE	93.8%	4.7/5.0 [*****]	EXCELLENT
#3	26BIT10319	Siddharth Patel	IT	77.5%	3.9/5.0 [****]	GOOD
#4	25BCE10521	Kabir Mehta	CIVIL	38.8%	1.9/5.0 [**]	FAIR
#5	25BME10115	Vikram Singh	MECH	33.8%	1.7/5.0 [*]	LOW

Overall 5-Star Rating Conversion Table

Match Percentage	Overall Rating	Star Representation	Harmony Tier
90.0% - 100.0%	4.5 - 5.0 / 5.0	[*****]	EXCELLENT
75.0% - 89.9%	3.8 - 4.4 / 5.0	[****]	GOOD
50.0% - 74.9%	2.5 - 3.7 / 5.0	[***]	MODERATE
35.0% - 49.9%	1.8 - 2.4 / 5.0	[**]	FAIR
Below 35.0%	0.0 - 1.7 / 5.0	[*]	LOW

11. Testing Approach & Automated Test Suite

A dedicated unit test suite (`test_system.py`) using Python's standard `unittest` framework verifies all components:

- **test_dictionary_lookup:** Verifies student retrieval using registration number dictionary keys.
- **test_tuple_structure_and_unpacking:** Validates immutable tuple packing and unpacking for demographic and habit records.
- **test_duplicate_registration_rejected:** Asserts dictionary key uniqueness prevents duplicate entries.
- **test_weight_mapping_completeness:** Asserts sum of weights mapping equals exactly 1.0 (100%).
- **test_sleep_and_cleanliness_logic:** Tests lifestyle distance scoring formulas for sleep, cleanliness, and diet.
- **test_overall_rating_and_tier_calculation:** Validates 5-star rating and harmony tier conversion accuracy.
- **test_compatibility_symmetry:** Proves mathematical symmetry (`Match(A, B) == Match(B, A)`).
- **test_sorting_order_of_predictions:** Validates descending sort order of candidate matches.
- **test_gender_isolation_in_matching:** Verifies male students match exclusively with male peers (and female with female).

Execution Result: Result: 9/9 unit tests passed in 0.037s (100% pass rate).

12. Challenges Faced & Solutions

- **1. Transition to 100% Real-Time Data:** Purged all pre-seeded mock records and reset operations so that the application authentically operates on live user input without hardcoded defaults.
- **2. State Persistence across Runs:** Handled file reading and writing dynamically so that records registered via command-line flags or menu sessions are instantly saved to disk.
- **3. Cross-Platform Console Encoding:** Replaced raw unicode star symbols with universal ASCII indicators `[*****]` to prevent Windows PowerShell cp1252 UnicodeEncodeError crashes.

13. Learnings & Key Takeaways

- **Collection Types Mastery:** Deepened understanding of Dictionary key lookups, hash collision avoidance, and Tuple immutability guarantees.
- **Modular Mappings Design:** Appreciated how dictionary mappings decouple configuration values from computational functions.
- **Algorithmic Sorting:** Learned how to customize Python's built-in `sorted()` with lambda functions for complex tuple sorting.
- **Test-Driven Verification:** Gained practical experience in test-driven development and writing automated test cases.

14. Future Enhancements

- **Graphical User Interface (GUI):** Developing a lightweight Tkinter or web-based UI for visual onboarding.
- **Advanced Filtering:** Allowing students to filter candidates by department or year of study.
- **Mutual Request Pairing:** Implementing Gale-Shapley stable matching between mutually preferred candidates.

15. References

- **1. Python Official Documentation - Built-in Types (Tuples, Dictionaries, Mapping Types) -** <https://docs.python.org/3/library/stdtypes.html>
- **2. Python Official Documentation - Sorting HOW TO -** <https://docs.python.org/3/howto/sorting.html>
- **3. Python Official Documentation - File Input and Output Tutorial -** <https://docs.python.org/3/tutorial/inputoutput.html>
- **4. VITyarthi Course Syllabus -** General Project Instructions & Submission Guidelines.